

TEMA 3. Excepciones

1. Empecemos un tema sobre control de errores en lenguajes de programación, con algo básico. En C, donde no existen las excepciones, pongamos un ejemplo de una raíz que toma número flotante positivo. Queremos controlar el error si la función recibe un número negativo. El usuario debe ser informado pero desde fuera de la función `raiz` ¿Cómo indicamos ese error?. Enumera dos opciones diferentes de diseñar, poniendo un ejemplo de código de cada una.

En java , la función `raiz` puede ser diseñada de varias formas para indicar un error cuando se recibe un número negativo. Una opción es devolver un valor especial, como `-1`, para indicar que se ha producido un error. Sin embargo, esta opción puede ser confusa, ya que `-1` podría ser un resultado válido para ciertos casos. Por ejemplo:

```
``java
public class Calculadora {

    public static double raiz(double numero) {

        if (numero < 0) {

            return -1; // Indica error

        }

        return Math.sqrt(numero);

    }

    public static void main(String[] args) {

        double resultado = raiz(-4);

        if (resultado == -1) {

            System.out.println("Error: número negativo no permitido.");

        } else {

            System.out.println("La raíz es: " + resultado);

        }

    }

}
```

...

Otra opción es lanzar una excepción personalizada para indicar el error. Esto permite manejar el error de manera más clara y específica. Por ejemplo:

```
```java
public class Calculadora {

 public static double raiz(double numero) throws IllegalArgumentException {
 if (numero < 0) {
 throw new IllegalArgumentException("Número negativo no permitido.");
 }
 return Math.sqrt(numero);
 }

 public static void main(String[] args) {
 try {
 double resultado = raiz(-4);
 System.out.println("La raíz es: " + resultado);
 } catch (IllegalArgumentException e) {
 System.out.println("Error: " + e.getMessage());
 }
 }
}
```
```

2. Brevemente ¿Qué es una ***excepción***? ¿Con qué objetivo las usa un programador cuando implementa funciones o cuando las llama?

Una excepción es un mecanismo que permite manejar errores o situaciones anómalas de manera controlada en un programa. Cuando se produce una situación que no se puede manejar dentro de una función, como un error de entrada o una condición inesperada, se puede lanzar una excepción para indicar que ha ocurrido un problema. El objetivo principal de las excepciones es separar el código normal del código de manejo de errores, lo que mejora la legibilidad y mantenibilidad del programa. Un programador puede usar excepciones para indicar que una función ha fallado debido a una condición específica, y luego el código que llama a esa función puede capturar la excepción para manejarla de manera adecuada, como mostrar un mensaje de error al usuario o intentar una solución alternativa. Esto permite que el programa continúe ejecutándose en lugar de detenerse abruptamente debido a un error no manejado.

3. Reescribe el mismo ejemplo de raíz, pero en Java, metiendo ese método en una clase `Calculadora` y llama a dicho método desde el método `main`, mostrando cómo se puede controlar desde fuera.

```
```java
public class Calculadora {

 public static double raiz(double numero) throws IllegalArgumentException {

 if (numero < 0) {

 throw new IllegalArgumentException("Número negativo no permitido.");

 }

 return Math.sqrt(numero);

 }

 public static void main(String[] args) {

 try {

 double resultado = raiz(-4);

 System.out.println("La raíz es: " + resultado);

 } catch (IllegalArgumentException e) {

 System.out.println("Error: " + e.getMessage());

 }

 }

}
```

...

## 4. ¿Qué es `throw` una excepción? ¿Qué es `try-catch` o `try-finally` una excepción? ¿Qué es que se `propague` una excepción? ¿Qué le va ocurriendo a las funciones en la pila de llamadas por donde se va propagando la excepción? ¿Las funciones que no la controlan se reanudan después de alguna forma? Explica con el mismo ejemplo anterior en Java de la raíz cuadrada.

Lanzar ( o `throw` una excepción es el acto de generar una señal de error o situación anómala en un programa. En el ejemplo de la raíz cuadrada, cuando se recibe un número negativo, se lanza una excepción `IllegalArgumentException` para indicar que la entrada no es válida. Controlar o capturar una excepción es el proceso de manejar esa señal de error en el código que llama a la función que lanzó la excepción. En el ejemplo, el bloque `try-catch` en el método `main` captura la excepción lanzada por el método `raiz` y maneja el error mostrando un mensaje al usuario.

Propagar una excepción significa que si una función no controla la excepción que ha lanzado, esta se transmite a la función que la llamó. En el ejemplo, si el método `raiz` lanza una excepción y no la captura, esta se propagará al método `main`, donde se captura en el bloque `catch`. Las funciones que no controlan la excepción no se reanudan después de lanzar la excepción; en cambio, el flujo del programa se interrumpe y se transfiere al bloque de manejo de excepciones más cercano en la pila de llamadas. En este caso, el método `raiz` no continúa ejecutándose después de lanzar la excepción, y el control se transfiere al bloque `catch` en el método `main` para manejar el error.

Ejemplo:

```
```java
public class Calculadora {

    public static double raiz(double numero) throws IllegalArgumentException {

        if (numero < 0) {

            throw new IllegalArgumentException("Número negativo no permitido."); // Lanzar la
            excepción

        }

        return Math.sqrt(numero);

    }

}
```

```

public static void main(String[] args) {
    try {
        double resultado = raiz(-4); // Llamar al método que puede lanzar una excepción
        System.out.println("La raíz es: " + resultado);
    } catch (IllegalArgumentException e) { // Capturar la excepción
        System.out.println("Error: " + e.getMessage());
    }
}
}
...

```

5. ¿Qué ventajas tiene frente a C, la *****propagación natural***** de las excepciones a través de la pila (*stack*) de llamadas?

La propagación natural de las excepciones a través de la pila de llamadas en Java ofrece varias ventajas frente a C. En primer lugar, permite una separación clara entre el código de manejo de errores y el código de lógica principal, lo que mejora la legibilidad y mantenibilidad del programa. En C, el manejo de errores a menudo se realiza mediante códigos de retorno o variables globales, lo que puede llevar a un código más confuso y propenso a errores. En segundo lugar, la propagación de excepciones en Java permite que los errores se manejen de manera más específica y localizada, ya que las excepciones pueden ser capturadas en el nivel adecuado de la pila de llamadas. En C, el manejo de errores a menudo requiere que el programador verifique manualmente los códigos de retorno en cada función, lo que puede ser tedioso y propenso a errores si se olvida verificar un código de error. En resumen, la propagación natural de las excepciones en Java facilita un manejo de errores más estructurado y eficiente en comparación con el enfoque tradicional de C.

6. En orientación a objetos, ¿las excepciones suelen ser objetos? ¿Qué ventajas tiene esto en términos de encapsulación? ¿Podemos entonces crear excepciones personalizadas?

En orientación a objetos, las excepciones suelen ser objetos que heredan de una clase base de excepciones. Esto tiene varias ventajas en términos de encapsulación. Al ser objetos, las excepciones pueden contener información adicional sobre el error que ha ocurrido, como un

mensaje descriptivo, un código de error o incluso la causa original del error. Esto permite que el código que maneja la excepción tenga acceso a esta información para tomar decisiones informadas sobre cómo manejar el error. Además, al ser objetos, las excepciones pueden ser personalizadas por el programador para adaptarse a las necesidades específicas de su aplicación. Por ejemplo, se pueden crear clases de excepciones personalizadas que representen errores específicos de la aplicación, lo que facilita la identificación y manejo de esos errores en particular. En resumen, el hecho de que las excepciones sean objetos en orientación a objetos proporciona una mayor flexibilidad y capacidad de encapsulación en el manejo de errores. Sí, es posible crear excepciones personalizadas en Java. Para hacerlo, se puede crear una nueva clase que extienda de una clase de excepción existente, como `Exception` o `RuntimeException`. Esto permite definir un nuevo tipo de excepción con su propia lógica y atributos personalizados. Por ejemplo:

```
```java

public class MiExcepcionPersonalizada extends Exception {

 public MiExcepcionPersonalizada(String mensaje) {

 super(mensaje);

 }

}

```
```

Luego, esta excepción personalizada se puede lanzar y capturar de la misma manera que cualquier otra excepción en Java, lo que permite un manejo de errores más específico y adaptado a las necesidades de la aplicación.

7. En relación con las ventajas de la encapsulación, comparando el ejemplo en C con Java. ¿Qué **información esencial lleva cualquier **objeto excepción** que es muy útil tener cuando se llega a un manejador?**

En Java, cualquier objeto excepción lleva información esencial que es muy útil para el manejo de errores. Esta información incluye un mensaje descriptivo que explica la naturaleza del error, lo que facilita la comprensión del problema por parte del programador o del usuario. Además, las excepciones en Java pueden contener una referencia a la causa original del error, lo que permite rastrear el origen del problema a través de la pila de llamadas. Esto es especialmente útil para depurar y solucionar problemas en el código. En contraste, en C, el manejo de errores suele depender de códigos de retorno o variables globales, lo que puede resultar en una falta de información detallada sobre el error y dificultar la identificación y resolución de problemas. En resumen, los objetos excepción en Java proporcionan una encapsulación más rica y

detallada de la información sobre los errores, lo que mejora significativamente el proceso de manejo de errores en comparación con C.

8. En Java, sobre el bloque `try-catch`, ¿se pueden tener más de un bloque `catch`? ¿cuántos bloques `catch` se ejecutan?

Sí, en Java se pueden tener más de un bloque `catch` para manejar diferentes tipos de excepciones que puedan ser lanzadas dentro del bloque `try`. Cada bloque `catch` está diseñado para capturar un tipo específico de excepción, y el programa ejecutará el primer bloque `catch` que coincida con el tipo de excepción lanzada. Si se lanza una excepción que no coincide con ningún bloque `catch`, la excepción se propagará hacia arriba en la pila de llamadas hasta que sea capturada por un bloque `catch` adecuado o hasta que alcance el nivel superior del programa, lo que resultará en la terminación del programa si no se maneja. En resumen, aunque se pueden tener múltiples bloques `catch`, solo uno de ellos se ejecutará para cada excepción lanzada, dependiendo del tipo de excepción.

Ejemplo:

```
``java
public class EjemploTryCatch {
    public static void main(String[] args) {
        try {
            int resultado = 10 / 0; // Esto lanzará una ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Error: División por cero.");
        } catch (Exception e) {
            System.out.println("Error: Ocurrió una excepción.");
        }
    }
}
...

```

9. Si las excepciones producen rupturas en el código llamador, ¿cómo podemos garantizar que se ejecuta siempre finalmente un código necesario para cierre de ficheros, liberación de recursos, antes de que continúe propagándose la excepción? Pon un ejemplo en Java con `finally`, tanto con `catch` como sin él.

En Java, se puede garantizar que se ejecute un código necesario para el cierre de ficheros o la liberación de recursos utilizando el bloque `finally`. El bloque `finally` se ejecuta siempre después de que el bloque `try` haya terminado, independientemente de si se lanzó una excepción o no. Esto significa que incluso si ocurre una excepción y se propaga hacia arriba, el código dentro del bloque `finally` se ejecutará antes de que la excepción continúe propagándose. Por ejemplo, si se abre un archivo dentro del bloque `try`, se puede asegurar que el archivo se cierre correctamente en el bloque `finally`, incluso si ocurre una excepción durante la lectura del archivo. Aquí hay un ejemplo en Java que muestra cómo usar `finally` tanto con `catch` como sin él:

```
```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class EjemploFinally {
 public static void main(String[] args) {
 BufferedReader reader = null;
 try {
 reader = new BufferedReader(new FileReader("archivo.txt"));
 String linea = reader.readLine();
 System.out.println("Contenido del archivo: " + linea);
 } catch (IOException e) {
 System.out.println("Error al leer el archivo: " + e.getMessage());
 } finally {
 try {
 if (reader != null) {
 reader.close(); // Asegura que el recurso se libere
 }
 } catch (IOException e) {
 System.out.println("Error al cerrar el archivo: " + e.getMessage());
 }
 }
 }
}
```



```

 }
 } catch (IOException e) {
 System.out.println("Error al cerrar el archivo: " + e.getMessage());
 }
}
}
}
}
...

```

En este ejemplo, el bloque `finally` garantiza que el `BufferedReader` se cierre correctamente, incluso si ocurre una excepción durante la lectura del archivo. Si no se captura la excepción de lectura, el bloque `finally` aún se ejecutará para cerrar el recurso antes de que la excepción continúe propagándose.

**## 10.** En Java, el bloque `finally` puede ir sin `catch`? ¿Se ejecuta siempre tanto si ocurre como si no ocurre una excepción? ¿Y si hay un `return` en medio del `try`?

Sí, en Java el bloque `finally` puede ir sin un bloque `catch`. El bloque `finally` se ejecutará siempre, independientemente de si ocurre una excepción o no. Incluso si hay un `return` dentro del bloque `try`, el bloque `finally` se ejecutará antes de que el método retorne el valor. Esto significa que cualquier código dentro del bloque `finally` se ejecutará antes de que el control regrese al llamador, lo que garantiza que se realicen las acciones necesarias, como liberar recursos o cerrar conexiones, incluso si el método está a punto de finalizar debido a un `return`. Aquí hay un ejemplo para ilustrar esto:

```

```java
public class EjemploFinallySinCatch {
    public static void main(String[] args) {
        System.out.println("Resultado: " + metodoConFinally());
    }

    public static int metodoConFinally() {
        try {
            System.out.println("Dentro del bloque try.");

```

```

        return 42; // El bloque finally se ejecutará antes de retornar este valor
    } finally {
        System.out.println("Dentro del bloque finally. Se ejecuta siempre.");
    }
}
}
...

```

En este ejemplo, el bloque `finally` se ejecutará incluso después de que el `return` en el bloque `try` haya sido alcanzado, lo que demuestra que el código dentro del bloque `finally` se ejecuta siempre, independientemente de las excepciones o de los retornos dentro del bloque `try`.

11. En Java, qué son las excepciones **""controladas""** y las **""no controladas""**? ¿Qué papel juega `RuntimeException`? Pon un ejemplo de excepciones típicas controladas y no controladas que incluso nosotros mismos podríamos usar. Haz dos listas con 3 o 4 ejemplos de situación donde se suele preferir una excepción controlada y donde se suele preferir una excepción no controlada.

En Java, las excepciones se clasifican en dos categorías: **""controladas""** y **""no controladas""**. Las excepciones controladas son aquellas que el programador debe manejar explícitamente, ya sea capturándolas con un bloque `catch` o declarando que el método puede lanzarlas con la cláusula `throws`. Estas excepciones suelen representar situaciones que pueden ser anticipadas y manejadas de manera razonable, como errores de entrada/salida o problemas de red. Por otro lado, las excepciones no controladas, también conocidas como excepciones de tiempo de ejecución, son aquellas que no requieren ser declaradas ni capturadas. Estas excepciones suelen indicar errores de programación, como errores de lógica o condiciones inesperadas, y se representan comúnmente por la clase `RuntimeException` y sus subclases. Ejemplos de excepciones controladas incluyen `IOException`, `SQLException` y `FileNotFoundException`, mientras que ejemplos de excepciones no controladas incluyen `NullPointerException`, `ArrayIndexOutOfBoundsException` y `IllegalArgumentException`.

Situaciones donde se suele preferir una excepción controlada:

1. Al manejar errores de entrada/salida, como al leer o escribir archivos.
2. Al interactuar con bases de datos, donde pueden ocurrir errores de conexión o consultas mal formadas.
3. Al realizar operaciones de red, donde pueden ocurrir problemas de conectividad o tiempo de espera.

Situaciones donde se suele preferir una excepción no controlada:

1. Al detectar errores de programación, como acceder a un índice fuera de los límites de un arreglo.
2. Al validar argumentos de métodos, donde se puede lanzar una `IllegalArgumentException` si los argumentos no cumplen con los requisitos.
3. Al manejar condiciones inesperadas que no se pueden anticipar, como una referencia nula que resulta en una `NullPointerException`. En estos casos, las excepciones no controladas pueden ayudar a identificar y corregir errores de programación de manera más rápida, ya que no requieren un manejo explícito por parte del programador.

12. ¿Qué es y para qué se usa `throws`? ¿Por qué es alternativa a capturar una excepción controlada?

`throws` es una cláusula en Java que se utiliza en la declaración de un método para indicar que ese método puede lanzar una o más excepciones. Al usar `throws`, el programador está declarando que el método puede generar ciertas excepciones y que el código que llama a ese método debe estar preparado para manejar esas excepciones, ya sea capturándolas con un bloque `catch` o propagándolas aún más hacia arriba en la pila de llamadas. La cláusula `throws` es una alternativa a capturar una excepción controlada dentro del mismo método, ya que permite que la responsabilidad de manejar la excepción recaiga en el código que llama al método, lo que puede ser útil cuando el manejo de la excepción es más apropiado en un nivel superior de la aplicación o cuando se desea mantener el código del método más limpio y enfocado en su lógica principal. En resumen, `throws` se usa para declarar que un método puede lanzar excepciones y para delegar la responsabilidad de manejarlas a los llamadores del método.

13. Pon un ejemplo en Java de firma de método que incluya `throws`, de una función que abre un fichero pero que declara que no le interesa manejar la excepción de si el fichero no existe, sino que se propague hacia arriba. Eso sí, acuérdate del `finally`.

```
```java
import java.io.BufferedReader;

import java.io.FileReader;

import java.io.IOException;
```

```

public class EjemploThrows {

 public static void main(String[] args) {

 try {

 String contenido = leerArchivo("archivo.txt");

 System.out.println("Contenido del archivo: " + contenido);

 } catch (IOException e) {

 System.out.println("Error al leer el archivo: " + e.getMessage());

 }

 }

 public static String leerArchivo(String nombreArchivo) throws IOException {

 BufferedReader reader = null;

 try {

 reader = new BufferedReader(new FileReader(nombreArchivo));

 return reader.readLine();

 } finally {

 if (reader != null) {

 reader.close(); // Asegura que el recurso se libere

 }

 }

 }

}

```

En este ejemplo, el método `leerArchivo` declara que puede lanzar una `IOException` utilizando la cláusula `throws`. Esto indica que el método no maneja la excepción de si el archivo no existe, sino que permite que se propague hacia arriba. El bloque `finally` se utiliza para garantizar que el recurso `BufferedReader` se cierre correctamente, incluso si ocurre una excepción durante la lectura del archivo. En el método `main`, se captura la excepción `IOException` para manejar cualquier error que pueda ocurrir al intentar leer el archivo. Este enfoque permite que el método `leerArchivo` se enfoque en su lógica principal de lectura de archivos, mientras que el manejo de errores se delega al código que llama al método, lo que

puede ser más apropiado en ciertos casos donde el manejo de la excepción es más relevante en un nivel superior de la aplicación.

## 14. ¿Podemos poner en `throws` excepciones no controladas, como `RuntimeException`? ¿Debería el método llamador entonces poner `try-catch` en ese caso? ¿Qué sentido tendría?

Sí, es posible poner en `throws` excepciones no controladas, como `RuntimeException`, aunque no es común ni recomendado hacerlo. Las excepciones no controladas son aquellas que indican errores de programación o condiciones inesperadas, y generalmente se espera que el programador las maneje de manera proactiva para corregir el código. Si un método declara que puede lanzar una excepción no controlada, el método llamador no está obligado a capturarla con un bloque `try-catch`, ya que estas excepciones suelen indicar errores que deben ser corregidos en lugar de manejados en tiempo de ejecución. Sin embargo, si el método llamador decide capturar la excepción no controlada, puede hacerlo para proporcionar una mejor experiencia al usuario o para registrar el error de manera más detallada antes de que el programa termine abruptamente. En general, declarar excepciones no controladas en `throws` puede ser útil para documentar que un método puede generar ciertos errores, pero se debe tener cuidado al hacerlo, ya que puede llevar a un código menos claro y a una mayor dificultad para identificar y corregir errores de programación. En resumen, aunque es técnicamente posible declarar excepciones no controladas en `throws`, no es una práctica común ni recomendada, y el manejo de estas excepciones generalmente debe centrarse en corregir el código en lugar de capturarlas en tiempo de ejecución.

## 15. ¿Cuándo se recomienda usar excepciones controladas, como `IOException`, y cuándo no controladas como `IllegalArgumentException`? ¿Existen en todos los lenguajes ambas opciones? En los que sólo existe una opción, ¿cuál es la más habitual?

Se recomienda usar excepciones controladas, como `IOException`, cuando se espera que el error pueda ser anticipado y manejado de manera razonable por el código que llama al método. Por ejemplo, al trabajar con archivos o redes, es común que ocurran errores de entrada/salida que pueden ser manejados de manera efectiva, como mostrar un mensaje de error al usuario o intentar una solución alternativa. Por otro lado, se recomienda usar excepciones no controladas, como `IllegalArgumentException`, para indicar errores de programación o condiciones inesperadas que no se pueden anticipar fácilmente. Estas excepciones suelen indicar problemas que deben ser corregidos en el código, como argumentos inválidos o referencias nulas. No todos los lenguajes de programación ofrecen ambas opciones de manejo de excepciones. En algunos lenguajes, como Python, todas las excepciones son no controladas y se espera que el programador las maneje adecuadamente. En otros lenguajes, como Java, se ofrece la opción de usar tanto excepciones controladas como no controladas. En los lenguajes donde solo existe una opción, la más habitual suele ser la de excepciones no controladas, ya que permite un manejo más flexible de los errores sin obligar al programador a

capturarlos explícitamente en cada caso. Sin embargo, esto también puede llevar a un código menos robusto si no se maneja adecuadamente, ya que los errores pueden propagarse sin ser capturados, lo que puede resultar en la terminación abrupta del programa. En resumen, la elección entre excepciones controladas y no controladas depende del contexto y del tipo de error que se espera manejar, así como de las características del lenguaje de programación utilizado.

## 16. ¿Tiene sentido lanzar excepciones dentro del `catch`? ¿Se puede relanzar la misma excepción capturada? ¿Cuándo tendría sentido hacer esto último? Pon ejemplos de ambos casos.

Sí, tiene sentido lanzar excepciones dentro del bloque `catch` en ciertas situaciones. Por ejemplo, si se captura una excepción y se determina que no se puede manejar adecuadamente en ese nivel, se puede lanzar una nueva excepción para indicar que el error persiste o para proporcionar información adicional sobre el error. Esto es especialmente útil cuando se desea encapsular una excepción de bajo nivel en una excepción personalizada de alto nivel que sea más relevante para el contexto de la aplicación. Por ejemplo:

```
```java
public class EjemploRelanzarExcepcion {

    public static void main(String[] args) {

        try {

            metodoA();

        } catch (Exception e) {

            System.out.println("Error en el método principal: " + e.getMessage());

        }

    }

    public static void metodoA() throws Exception {

        try {

            metodoB();

        } catch (IOException e) {

            // Relanzar la misma excepción capturada

            throw e; // Esto permite que la excepción se propague hacia arriba

        }

    }

}
```

```

    }
}

public static void metodoB() throws IOException {

    throw new IOException("Error de entrada/salida en metodoB");

}

}

...

```

En este ejemplo, el método `metodoA` captura una `IOException` lanzada por `metodoB`, pero en lugar de manejarla, la relanza para que el método principal pueda capturarla y manejarla de manera más adecuada. Esto tiene sentido cuando el método que captura la excepción no tiene suficiente contexto para manejarla correctamente, y se desea que el error sea manejado en un nivel superior de la aplicación donde se pueda tomar una decisión informada sobre cómo proceder.

17. ¿En qué consiste que una excepción sea la **causa** de otra excepción? Pon un ejemplo en Java, donde capturemos una excepción de bajo nivel y la encapsulemos en otra personalizada de alto nivel. Cuando una excepción sale por pantalla y tiene una causa, ¿se ve?

Una excepción puede ser la "causa" de otra excepción cuando una excepción de bajo nivel se captura y se encapsula dentro de una nueva excepción personalizada de alto nivel. Esto se hace para proporcionar un contexto más relevante sobre el error que ha ocurrido, especialmente cuando el error original no es lo suficientemente descriptivo o relevante para el nivel de la aplicación donde se maneja la excepción. En Java, esto se puede lograr utilizando el constructor de la clase de excepción personalizada que acepta una causa como argumento. Cuando una excepción con una causa se imprime en la consola, se muestra tanto el mensaje de la excepción principal como el mensaje de la causa, lo que ayuda a entender mejor el origen del error. Aquí hay un ejemplo en Java:

```

```java
public class EjemploExcepcionConCausa {

 public static void main(String[] args) {

 try {

 metodoA();

 } catch (MiExcepcionPersonalizada e) {

```

```

 System.out.println("Error en el método principal: " + e.getMessage());

 System.out.println("Causa del error: " + e.getCause().getMessage());
 }
}

public static void metodoA() throws MiExcepcionPersonalizada {
 try {
 metodoB();
 } catch (IOException e) {
 // Encapsular la excepción de bajo nivel en una excepción personalizada de alto nivel
 throw new MiExcepcionPersonalizada("Error en metodoA", e);
 }
}

public static void metodoB() throws IOException {
 throw new IOException("Error de entrada/salida en metodoB");
}
}
...

```

En este ejemplo, el método `metodoA` captura una `IOException` lanzada por `metodoB` y la encapsula dentro de una nueva excepción personalizada llamada `MiExcepcionPersonalizada`. Al imprimir la excepción en el método principal, se muestra tanto el mensaje de la excepción personalizada como el mensaje de la causa original, lo que proporciona una visión más completa del error que ha ocurrido.